

3

Advanced C Programming: Functions, Conditionals, and Loops

Objectives:

Some of the advanced elements of C include functions, conditionals, and loops. When you complete this lab, you should be able to:

- Analyze a set of requirements for a problem and create suitable functions.
- Create C statements to conditionally operate on certain values
- Create C statements to repeat a set of code multiple times
- Comment a program for understandability.
- Create a project using the standard 3-file organization

Related material to read:

- C Text, Chapters 3-5 in General
- Appendix A, ASCII Character Codes

3.1 Introduction

Today will be the first two-week lab! Lab reports will be due a week after the second week of this lab. This lab will go over standard file organization in C, conditional statements, commenting, and loops.

3.2 C File Organization: 3-File Format

In most standard C programs, the code is organized into 3 types of files:

- 1) **Main File:** We have already seen this extensively. This file serves as the main entry point into a C program (main function). This file typically governs the high-level functionality of the program, and mainly consists of calling function (more on where functions go in the next bullet) and manipulating variables.
- 2) **(Other) Source Files:** Typically, we have many different functions in a single C program. Some of these functions may involve calculating BMI, or perhaps converting a digit character to its integer representation (`char2int`). It is common practice to group up these functions and place them in separate `.c` files when deemed appropriate. For example, we could place all the bmi calculation functions into a file `"bmi.c"`.
- 3) **Header Files:** For each of these source files (besides main), it is commonplace to have an associated header or `.h` file. For example, `"bmi.c"` would have an accompanying `"bmi.h"`. These header files are used as an interface between the different `.c` files. This header file is useful to centralize the definitions and declarations shared among files so that only one copy is created that is shared among all other source files. We will discuss this a bit further.

In this lab, we will create `"prob3a.c"` and `"prob3a.h"` files for the different problems in order to define and use the functions you create for each problem.

3.3 Header Files: Function Prototypes and Guard-bands

When using multiple files in C, the compiler takes each file and individually breaks them down to object files (mostly machine code, what the computer uses to actually execute programs). To allow a program to utilize functions across different object files, header files are used to define "function prototypes." A function prototype is simply the declaration of a function that specifies the function's name, parameters, and return type. It doesn't contain the function body. A function prototype gives information to the compiler that the function may later be used in the program. The general syntax is:

```
returnType functionName(type1 argument1, type2 argument2, ...);
```

For example, inside `"prob3a.h"` we may have the function prototype:

```
int char2int (char digit);
```

This is an example of a function prototype for the function `char2int`. **NOTE: don't forget the semicolon at the end of function prototypes.** Then, if we want to use `char2int` from any source file, all we have to do is add a preprocessor include statement to the top of that source file `"#include "prob3a.h"`. We must also put that same include statement at the top of the corresponding source file `"prob3a.c"`.

Header Guard-Bands

Now that we have function prototypes explained, it is common for multiple source files to include a particular header and its corresponding source code. In order to prevent the same header file from being included multiple times and causing errors, inclusion guards are utilized. For example, at the top of the “prob3a.h”, there are two preprocessor statements:

```
#ifndef __PROB3A_H__
#define __PROB3A_H__
```

And at the bottom we have another preprocessor statement:

```
#endif /* __PROB3A_H__ */
```

This essentially includes everything between `#ifndef` and `#endif` only when `__PROB3A_H__` is not defined (that’s what `ndef` means). Since we define `__PROB3A_H__` in the following line, this guarantees that the contents in the header file will only be included the first time we go through it.

3.4 Review: Functions

To clarify some terminology, functions are a group of statements that perform a task. Typically, these functions require some parameters (technical term for inputs) that are used in these statements to perform the task. Since we may want to perform this task multiple times, we can call (technical term for use) this function whenever needed. To call a function, we simply use its function name with any required parameters. To explain visually, the following is the general format of a function.

```
returnType functionName(paramType1 param1, paramType2
                        param2,...,paramTypeN paramN)
{
    // Function Body
    return returnValue;
}
```

The `returnType` specifies the type of the `returnValue`, the `functionName` can be anything reasonable that describes the function’s task, and each `paramType` specifies the type of the param. A simple example:

```
double calcCircumference (double radius)
{
    return (2*PI*radius);
}
```

Above is a function called `calcCircumference` that calculates the circumference using an input parameter and returns the result as a double type. An example call (use):

```
double circumference = calcCircumference(10);
```

3.5 Review: Control Statements

Please see Lecture 5 for a more detailed discussion. However, something to keep in mind, think about which control statement makes the most sense in each instance. Should I use an “if”? or a “switch”? Similarly, with “for” vs. “while” vs. “do-while”.

3.6 Problem 3a: Calculating Body Mass Index (BMI) using Functions

In the first problem of this lab, we practice creating functions and using the 3-file format/organization. In lab 2, you should have completed a solution to the body mass index (BMI) for a given height and weight calculation problem. This time, we will rewrite this solution to use functions. Please find the following description of the problem from Lab 2:

BMI is a general measurement that can help identify if a person is underweight, a healthy weight, or overweight. Your program is required to prompt the user for weight in pounds and height in feet. The height must be converted into inches (recall: 1 foot = 12 inches). Once the BMI has been calculated, display the resultant BMI value. Use the equation below to calculate the BMI.

$$\text{BMI} = ((\text{weight in pounds})/(\text{height in inches})^2) * 703$$

First, we need to analyze the problem and determine subproblems. Second, we need to draw a structure chart to show the relationships between subproblems. The original problem is to compute BMI.

Original Problem – Level 0

Compute BMI

In order to compute the BMI, we need to get inputs from the user for weight (in pounds) and height (in feet).

Subproblems – Level 1

Get Height

Get Weight

We also need to convert the height in feet to inches, calculate the BMI based on given user inputs, and display the BMI. Converting height in feet to inches is really a detailed subproblem of calculating the BMI, so it should be placed in level 2 of the chart.

Subproblems – Level 1 Cont.

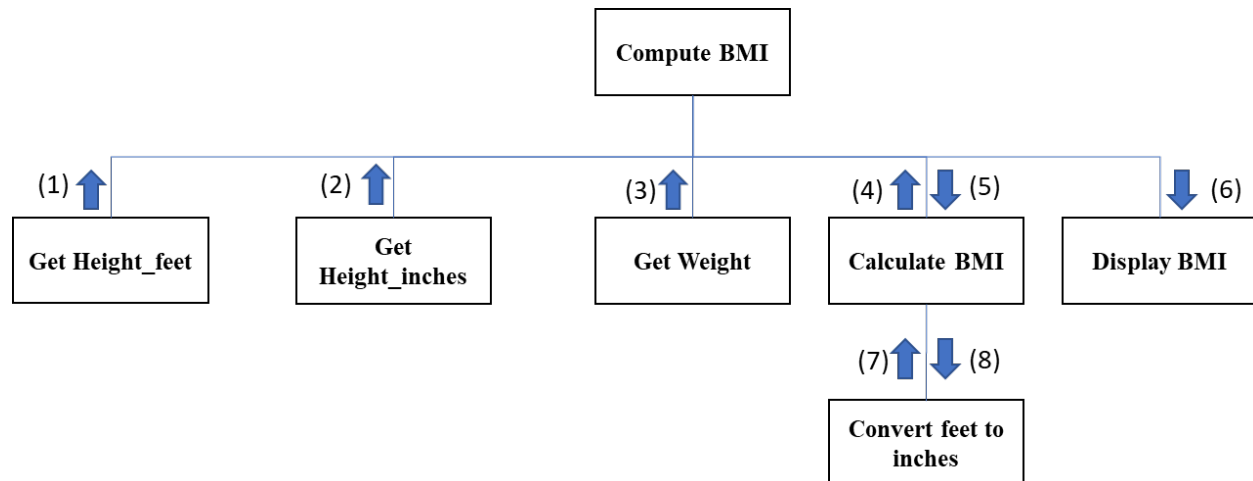
Display BMI

Calculate BMI

Detailed Subproblems – Level 2

**Convert feet to
inches**

The complete structure chart with data flow information is listed below:



Data flow:

- (1) The feet portion of the height
- (2) The inches portion of the height
- (3) Weight in pounds
- (4) BMI
- (5) Height in feet and inches, Weight in pounds
- (6) BMI
- (7) Height in inches
- (8) Height in feet and inches

Procedure

As always, create a Platform.IO project using the same procedures as Lab 1. The only difference is that you should use `main3a.c` instead of `main.c` and include both `prob3a.c` and `prob3a.h`. The following text in blue is copied from Lab 2 as a reminder. Step 3 and on are new.

1. Prompt the user to input the height in feet and inches (ask for each separately) and the weight in pounds. Use `char2int` from problem 1 to convert the characters to integers.
 - a. Assume that both feet and inches for height can take a value between 0-9. I know this excludes 10 and 11 inches (5'11") but to simplify the problem (need more advanced concepts to include them) ignore them for now.
 - b. Assume that weight can be from 100 to 999. This guarantees that weight will always be three digits. In order to read in three digits, just use `UARTgetc` three times. The first call to `UARTgetc` should get the 100's digit, the second call to `UARTgetc` should get the 10's digit, and the last call to `UARTgetc` should get

the 1's digit. Convert each digit to its integer and combine them to get the weight.

2. Calculate BMI and print out the value. Don't forget to cast!
3. Create the following functions in prob3a.c. I have already created display_bmi for you as an example:
 - a. int get_weight (void) // prompts the user for weight in pounds, and returns the weight
 - b. int get_height_feet (void) // prompts the user for height in feet, and returns the height
 - c. int get_height_inches (void) // prompts the user for remaining height in inches, and returns the height
 - d. int convert_height_to_inches (int height_feet, int height_inches) // converts the height of the user from feet to inches, and returns the height in inches
 - e. double calculate_bmi (int weight_in_pounds, int height_feet, int height_inches) // evaluates the BMI based on weight in pounds and height in inches, and returns the BMI – call convert_height_to_inches ()
4. Follow the comment example for display_bmi and comment all functions. Similarly, modify the comments at the top of main3a.c and prob3a.c to match your details.
5. Instead of using two different functions to return the feet and inches of the height, we can use pointers to modify them both at the same time.

```
void get_height(int *height_feet, int *height_inches)
```

By inputting the pointer (address) as the input parameter, we can modify the original variable by dereferencing (*) the pointer. Remember, we can get the address of a variable using the & operator. Create and define the get_height function. You'll need to uncomment the function prototype in the prob3a.h file.

Call the lab TA and demonstrate the program working. The TA will enter their own test inputs, if your program outputs the correct values the lab TA will check you off. The TA will also scan through your functions to see if they were done correctly.

Example Output:

```

--- Available filters and text transformations: colorize, debug, default
rintable, send_on_enter, time
--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
Hello, Lab 3!
Problem a

Enter Height: (feet): 5
Enter Height: (inches): 7
Enter Weight: (3 digits): 151
BMI: 23.647360

```

3.7 Problem 3b Daily Calorie Expenditure Calculator

In the second problem of this lab, we will continue practicing creating functions, using the 3-file format/organization, and using conditional statements. This lab will write a Calorie Calculator program to determine what a person's caloric intake should be for a 24-hour span. The program must make a decision about caloric intake, to maintain current body weight, based on gender, age, weight, and height, and activity level. To determine total daily calorie needs, we first must compute the Basal Metabolic Rate (BMR). The BMR formula is the following:

Women: $\text{BMR} = 655 + (4.35 * \text{weight in pounds}) + (4.7 * \text{height in inches}) - (4.7 * \text{age in years})$

Men: $\text{BMR} = 66 + (6.23 * \text{weight in pounds}) + (12.7 * \text{height in inches}) - (6.8 * \text{age in years})$

To determine the total daily calorie needs, your program must apply the following guidelines:

1. Sedentary (little to no exercise): $\text{Calories} = \text{BMR} * 1.2$
2. Low activity: $\text{Calories} = \text{BMR} * 1.375$
3. Moderate activity: $\text{Calories} = \text{BMR} * 1.55$
4. High activity: $\text{Calories} = \text{BMR} * 1.725$
5. Extra activity: $\text{Calories} = \text{BMR} * 1.9$

Procedure

To start, create a Platform.IO project using the same procedures as Lab 1. The only difference is that you should use main3b.c instead of main.c and include both prob3b.c and prob3b.h. Define functions where appropriate! Prompt the user for age (assume 10-99), gender, activity level, weight, and height values. Reuse functions from problem 3a when you can! Output the result. Comment all functions. Similarly, modify the comments at the top of main3b.c and prob3b.c to match your details.

Some details:

- You can use 'M' or 'F' for gender input.
- Use a switch statement on activity level to calculate final calories.

Call the lab TA and demonstrate the program working. The TA will enter their own test inputs, if your program outputs the correct values the lab TA will check you off. The TA will also scan through your functions to see if they were done correctly.

Example Output:

```
--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
Hello, Lab 3!
Problem b

Enter Height: (feet): 5
Enter Height: (inches): 8
Enter Weight: (3 digits): 155
Enter Age: (2 digits): 19
Enter Gender: (M/F): F
For the following -
0: Sedentary
1: Low
2: Moderate
3: High
4: Extra High
Enter Activity Level (0 to 4): 2
BMR = 1559.550000
Calculated Daily Calories = 2417
```

3.8 Problem 3c GPA Calculator

In the third problem of this lab, we will continue practicing creating functions, using the 3-file format/organization, using conditional statements, and loops. This lab will write a GPA calculator program to determine the GPA of a person who has taken five classes.

Grades to Grade Points

- A = 4 points
- B = 3 points
- C = 2 points
- D = 1 points
- F = 0 points

Procedure

To start, create a Platform.IO project using the same procedures as Lab 1. The only difference is that you should use `main3c.c` instead of `main.c` and include both `prob3c.c` and `prob3c.h`. Define functions where appropriate! Comment all functions. Similarly, modify the comments at the top of `main3c.c` and `prob3c.c` to match your details.

1. Ask for the grades and credits of five classes. Place each grade and credit into a separate variable (e.g., `char grade1`, `char grade2`, `int credit1`, `int credit2`, etc.).
2. Calculate the amount of grade points for each class (switch statement is appropriate here)
3. Calculate the total sum of credits.
4. Calculate the GPA using (2) and (3).
5. Instead of using separate variables for each class, reimplement this problem using arrays (e.g., `char grade[5]`). You can use pointer notation for arrays if you pass them into functions (Remember that arrays are essentially a pointer to the first element).

Call the lab TA and demonstrate the program working. The TA will enter their own test inputs, if your program outputs the correct values the lab TA will check you off. The TA will also scan through your functions to see if they were done correctly.

Example Output:

```
--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
Hello, Lab 3!
Problem c

Enter credits for class 1 (one digit): 3
Enter letter grade for class 1: A
Enter credits for class 2 (one digit): 2
Enter letter grade for class 2: D
Enter credits for class 3 (one digit): 4
Enter letter grade for class 3: B
Enter credits for class 4 (one digit): 3
Enter letter grade for class 4: A
Enter credits for class 5 (one digit): 2
Enter letter grade for class 5: C
Total Grade Points: 42
Total Credits: 14

GPA: 3.000000
```

3.9 Problem 3d Error Checking

In the fourth problem of this lab, we will continue practicing using conditional statements and loops. For all user inputs in problems 3a – 3c, do error checking (on an erroneous input, the program should print an error and ask the user again for the input). For example, in problem 3b, if the user enters ‘D’ for gender, the program should print out “ERROR: D is an invalid input” then print out “Gender (M/F): ” again. (10 pts) **What is the most appropriate loop here?**

Call the lab TA and demonstrate the program working. The TA will enter their own test inputs, if your program outputs the correct values the lab TA will check you off. The TA will also scan through your functions to see if they were done correctly. The TA will do this once for 3a, 3b, and 3c.

3.10 Questions:

1. (10 pts) How would you modify problem 3a to accept input with metric measurements (i.e., height in cm and weight in Kg)? You don’t have to write code, just provide the steps and calculations needed to convert from metric input to the english system.
2. Please rate (1-10): Lab content (interesting?), enough time to complete content in lab? Any comments?

3.11 Prework:

Look through all the source files supplied in ECE_Lab3_files.zip. Show a list of all of the source files in the following 4 categories: Main Files, Other C Source Files, Header Files, and Other sources.

3.12 Lab report:

The lab report is due at 11:59PM one week after the conclusion of this lab. Since this is a two-week lab, the lab report will be due two weeks after the first lab session. Monday labs are due on Monday, Tuesday labs are due on Tuesday, and Friday labs are due on Friday. For the lab write up, include the following:

1. A copy of your working .c/.h files for the program. (All .c/.h files) – 40 pts
2. A brief discussion of the objectives of the lab and the procedures performed in the lab. Includes checkoffs (PDF) – 40 pts
3. Answers to any **questions** in the discussion, procedure, or question sections of the lab. (PDF) – 20 pts

Submit it to Canvas (write up separate from the working files, do not zip files together). Please keep in mind that you must have your work checked off (look for the green sections) by the TA to receive credit for the lab.

Appendix A: ASCII Table

Decimal - Binary - Octal - Hex – ASCII
Conversion Chart

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	"	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	\$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	'	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL